

REPORTE DE SEGURIDAD DE SOFTWARE

Modelo de Amenazas · Análisis SAST · Análisis DAST

MedTriage – Sistema de Gestión de Turnos y Triage Médico Automatizado

1. Introducción

El presente documento constituye el Reporte de Seguridad de Software del sistema MedTriage. Al ser un proyecto académico en fase de documentación — sin código fuente implementado — el análisis adopta el enfoque proyectado: se modelan las amenazas con base en la arquitectura definida en el SAD (Spring Boot + PostgreSQL + HTML/CSS/JS) y se documentan los procedimientos de análisis SAST y DAST que deben ejecutarse cuando el código sea implementado.

MedTriage maneja datos clínicos sensibles de pacientes en un servicio de urgencias, lo que lo clasifica como un sistema de información de salud sujeto a la Ley 1581 de 2012 (Habeas Data) y a los estándares de seguridad de datos médicos. Un fallo de seguridad en este sistema puede comprometer la privacidad de los pacientes y, en el caso del algoritmo de triaje, poner en riesgo su integridad física.

Este reporte está alineado con:

- SRS IEEE 830: RNF-001 (AES-256), RNF-002 (HTTPS), REST-001 (Ley 1581 de 2012).
- SAD: arquitectura MVC, API REST, capas Frontend/Backend/PostgreSQL.
- Manual de Mantenimiento: Escenario A (caída BD), Escenario C (bloqueo de cuentas), DEU-004 (2FA).
- Matriz de Riesgos (MedTriage_Gestion): R03 (incumplimiento Ley 1581), R08 (pérdida de datos), R09 (errores en clasificación de triaje).

2. Superficie de Ataque

La superficie de ataque de MedTriage comprende todos los puntos de entrada y salida del sistema accesibles desde el exterior. Se identifican tres zonas de exposición según la arquitectura C4 del SAD:

Zona de exposición	Descripción y activos expuestos
Zona 1 – Interfaz Web (Frontend)	Formularios de login, registro de paciente, captura de signos vitales, discriminadores Manchester y evolución clínica. Accesible desde navegador web estándar. Superficie más expuesta a ataques XSS, CSRF y fuerza bruta.
Zona 2 – API REST (Backend Spring Boot)	Endpoints HTTP expuestos: /api/auth/login, /api/pacientes, /api/signos-vitales, /api/triaje, /api/cola, /api/turnos/pantalla-publica, /api/reportes. Superficie crítica: procesa datos clínicos y ejecuta el Algoritmo Manchester.
Zona 3 – Base de Datos (PostgreSQL)	Accesible únicamente desde el backend (no expuesta directamente al exterior). Almacena datos clínicos, usuarios con hashes de contraseñas, niveles de triaje y registros de auditoría. Riesgo principal: acceso físico al servidor.

Actores externos	Personal médico y de enfermería (usuarios legítimos), pacientes y acompañantes (pantalla pública sin autenticación), atacantes externos (red LAN o internet si el servidor es accesible), atacantes internos (personal con credenciales válidas).
-------------------------	---

3. Modelo de Amenazas – Metodología STRIDE

Se aplica la metodología STRIDE (Microsoft Threat Modeling) para identificar sistemáticamente las amenazas del sistema MedTriage. Cada amenaza se clasifica por probabilidad, impacto y nivel de riesgo resultante (Probabilidad × Impacto).

Probabilidad Alta	Amenaza realista con técnicas conocidas y acceso probable.
Probabilidad Media	Amenaza factible pero que requiere conocimiento técnico o acceso específico.
Probabilidad Baja	Amenaza posible pero con múltiples barreras de protección o acceso restringido.
Nivel CRÍTICO	Probabilidad Alta + Impacto Alto. Requiere mitigación inmediata (Sprint 1).
Nivel ALTO	Probabilidad Media + Impacto Alto. Requiere mitigación en el sprint correspondiente.
Nivel MEDIO	Probabilidad Media + Impacto Medio. Programar en backlog de seguridad.

ID	Superficie	Amenaza	Prob.	Impacto	Nivel
S – Suplantación de Identidad (Spoofing)					
AM-S01	Login web (RF-001)	Ataque de fuerza bruta sobre el formulario de inicio de sesión para adivinar credenciales del personal médico.	Alta	Alta	CRÍTICO
AM-S02	Token JWT (Backend)	Robo o falsificación del token JWT para suplantar el rol de Administrador y acceder al módulo de reportes o gestión de usuarios.	Media	Alta	ALTO
AM-S03	Sesión persistente	Secuestro de sesión (session hijacking) si el token JWT no tiene tiempo de expiración o no se invalida al cerrar sesión.	Media	Alta	ALTO
T – Manipulación de Datos (Tampering)					
AM-T01	API REST (Backend)	Manipulación de parámetros en la petición POST /api/triage para alterar el nivel de triaje calculado por el Algoritmo Manchester.	Media	Alta	ALTO
AM-T02	Base de datos PostgreSQL	Inyección SQL en los endpoints de registro de paciente o signos vitales si los parámetros no	Baja	Alta	ALTO

		están correctamente parametrizados.			
AM-T03	Pantalla pública (RF-008)	Manipulación del endpoint GET /api/turnos/pantalla-publica para mostrar información clínica no anonimizada en la sala de espera.	Baja	Media	MEDIO
R – Repudio (Repudiation)					
AM-R01	Modificación de triaje (RF-006)	Un enfermero modifica manualmente el nivel de triaje sin dejar registro de justificación, negando posteriormente haber realizado el cambio.	Baja	Media	MEDIO
AM-R02	Acceso de Administrador	El Administrador realiza acciones sobre cuentas de usuario (activar/desactivar) sin que quede registro de auditoría en el sistema.	Baja	Media	MEDIO
I – Divulgación de Información (Info Disclosure)					
AM-I01	Datos clínicos en tránsito	Interceptación de datos clínicos transmitidos en texto plano si el protocolo HTTPS no está correctamente configurado (RNF-002).	Baja	Alta	ALTO
AM-I02	Datos en reposo (BD)	Acceso físico al servidor hospitalario y lectura de datos clínicos sin cifrar en la base de datos PostgreSQL (RNF-001, Ley 1581/2012).	Baja	Alta	ALTO
AM-I03	Mensajes de error verbosos	Los mensajes de error del backend Spring Boot revelan detalles de la arquitectura interna (stack traces, rutas de archivos) al usuario no autorizado.	Media	Media	MEDIO
AM-I04	Endpoint pantalla pública	La pantalla de turnos expone información que permite identificar al paciente (nombre completo en lugar de iniciales o código de turno).	Media	Media	MEDIO
D – Denegación de Servicio (DoS)					
AM-D01	API REST (Backend)	Sobrecarga de peticiones concurrentes al endpoint /api/triaje que agota el pool de conexiones HikariCP y provoca	Media	Alta	ALTO

		caída del servicio (Escenario A del Manual de Mantenimiento).			
AM-D02	Base de datos	Consultas masivas sin paginación al endpoint GET /api/pacientes generan timeouts y degradan el rendimiento del sistema (DEU-003 del Manual de Mantenimiento).	Media	Media	MEDIO
E – Escalación de Privilegios (Elevation of Privilege)					
AM-E01	Endpoints sin control de rol	Un usuario con rol Admisión accede directamente a la URL /api/medico/cola o /api/admin/reportes si los endpoints no verifican el JWT y el rol del solicitante.	Media	Alta	ALTO
AM-E02	Cuenta bloqueada reactivada sin verificación	El Administrador reactiva una cuenta sin verificar la identidad del solicitante, permitiendo a un atacante recuperar acceso a una cuenta comprometida (Escenario C del Manual de Mantenimiento).	Baja	Alta	ALTO

4. Controles de Mitigación por Amenaza

Contramedidas técnicas alineadas con la arquitectura del SAD

La siguiente tabla define el control técnico de mitigación para cada amenaza identificada, indicando la capa arquitectónica responsable de su implementación y el requisito del SRS al que responde.

Amenaza	Vector	Control de mitigación	Capa responsable	Requisito
AM-S01	Fuerza bruta login	Implementar bloqueo de cuenta tras 5 intentos fallidos (ya modelado en CU-001, Escenario C del Manual). Añadir CAPTCHA o rate-limiting en el endpoint /api/auth/login.	Spring Security	DEU-004 (2FA)
AM-S02/S03	Robo/falsificación JWT	Configurar expiración corta del token JWT (≤ 8 horas). Usar firma HS256 con clave secreta de mínimo 256 bits almacenada en variable de entorno, no en código.	Spring Security	RNF-001
AM-T01	Manipulación parámetros API	Validar y sanear todos los parámetros de entrada en el backend con Bean Validation (@Valid) antes de ejecutar la lógica de negocio.	Spring Boot	RF-005

AM-T02	Inyección SQL	Usar exclusivamente JPA/Hibernate con consultas parametrizadas (JPQL / Criteria API). Prohibir concatenación de strings en consultas SQL.	PostgreSQL + JPA	RNF-001
AM-I01	Datos en tránsito	Habilitar HTTPS/TLS 1.3 en el servidor. Redirigir automáticamente HTTP → HTTPS. Configurar HSTS en los headers de respuesta del backend.	DevOps / Backend	RNF-002
AM-I02	Datos en reposo	Cifrar columnas clínicas sensibles con AES-256 mediante extensión pgcrypto de PostgreSQL o cifrado a nivel de aplicación en Spring Boot.	Backend / DevOps	RNF-001
AM-I03	Errores verbosos	Configurar ExceptionHandler global en Spring Boot que retorne mensajes de error genéricos al cliente. Los detalles técnicos solo se registran en el log del servidor.	Backend	RNF-010
AM-R01	Repudio en triaje	El CU-006 ya obliga a registrar justificación con usuario, fecha y hora (trazabilidad). Extender con log de auditoría inmutable para todas las modificaciones clínicas.	Backend	RF-006
AM-D01	DoS por sobrecarga API	Implementar rate-limiting por IP en Spring Boot. Ajustar maximumPoolSize de HikariCP según la carga esperada (definido en Manual de Mantenimiento, Escenario A).	Backend / DevOps	RNF-003
AM-D02	DoS por consultas masivas	Implementar paginación en GET /api/pacientes (DEU-003). Añadir índices en las columnas de filtrado frecuente en PostgreSQL.	Backend	RNF-005/006
AM-E01	Escalación de privilegios	Aplicar @PreAuthorize con el rol requerido en cada endpoint REST. Verificar que ningún endpoint quede expuesto sin middleware de seguridad de Spring Security.	Backend	RF-001
AM-E02	Reactivación sin verificación	El procedimiento del Manual de Mantenimiento (Escenario C) ya exige verificación presencial. Añadir log de auditoría con timestamp en cada cambio de estado de usuario.	Administrador	REST-001

5. Análisis SAST – Static Application Security Testing

Vulnerabilidades proyectadas en el código fuente de MedTriage

5.1 Herramientas propuestas

Herramienta	Uso en MedTriage
SonarQube (Security Hotspots)	Detección de hotspots de seguridad integrada al pipeline CI. Cubre Java/Spring Boot y JavaScript.
OWASP Dependency-Check	Escaneo de dependencias Maven (pom.xml) para detectar librerías con CVEs conocidos. Integrado en el build de Maven.
SpotBugs + Find Security Bugs	Plugin Java que detecta vulnerabilidades específicas: SQL injection, path traversal, deserialización insegura.
ESLint (Security plugin)	Análisis estático del código JavaScript del frontend para detectar XSS y uso inseguro de eval().

5.2 Vulnerabilidades proyectadas y requeridas (código sin implementar)

La siguiente tabla documenta las vulnerabilidades que el SAST debe detectar y resolver durante el desarrollo de MedTriage, basándose en los patrones de riesgo comunes del stack tecnológico definido en el SAD (Spring Boot + PostgreSQL + HTML/CSS/JS):

ID	Vulnerabilidad	Severidad	Descripción y condición de detección	Acción requerida
SAST-01	Inyección SQL / JPQL	CRÍTICO	Detectado si se encuentra concatenación de strings en consultas a la base de datos. Spring Boot con JPA parametrizada elimina este riesgo.	Corregir antes de Sprint 1 – Sprint 1
SAST-02	Hardcoded credentials	CRÍTICO	Claves de base de datos, secretos JWT o certificados SSL escritos directamente en el código fuente o en application.properties versionado.	Mover a variables de entorno – Sprint 1
SAST-03	Exposición de stack trace	ALTO	El backend retorna stack traces completos en respuestas HTTP 500. Detectado por análisis de los ExceptionHandler de Spring Boot.	Implementar GlobalExceptionHandler – Sprint 1
SAST-04	Token JWT sin expiración	ALTO	El token JWT no configura el campo exp (expiration). Un token robado tendría validez indefinida.	Configurar expiración $\leq$ 8 h – Sprint 1
SAST-05	Endpoints sin @PreAuthorize	ALTO	Métodos del controlador REST sin anotación de seguridad accesibles para cualquier usuario autenticado, independientemente de su rol.	Auditar todos los endpoints – Sprint 2
SAST-06	Dependencias desactualizadas (CVE)	MEDIO	Versiones de Spring Boot o PostgreSQL JDBC driver con vulnerabilidades conocidas	Revisión cada 6 meses según Plan de Parches

			publicadas después de la versión base del proyecto.	
SAST-07	Cross-Site Scripting (XSS) – Frontend	MEDIO	Inserción de contenido HTML no sanitizado en la pantalla pública de turnos o en el formulario de evolución clínica.	Sanitizar inputs en Frontend – Sprint 3
SAST-08	CSRF en formularios Frontend	BAJO	Formularios HTML sin token CSRF que permitan peticiones cross-site. Spring Security incluye protección CSRF por defecto.	Verificar configuración CSRF – Sprint 1

6. Análisis DAST – Dynamic Application Security Testing

Pruebas de vulnerabilidades sobre la aplicación en ejecución

6.1 Herramienta propuesta: OWASP ZAP

El DAST se ejecuta sobre la aplicación MedTriage en ejecución en el entorno de staging, utilizando OWASP ZAP (Zed Attack Proxy). A diferencia del SAST, el DAST no analiza el código fuente sino el comportamiento real de la aplicación frente a peticiones maliciosas.

Parámetro	Valor definido
Herramienta	OWASP ZAP 2.x (Community Edition)
Entorno de ejecución	Staging (réplica del entorno de producción, jamás en producción)
Tipo de escaneo	Escaneo activo (Active Scan) + escaneo de spider sobre todos los endpoints de la API REST
URL base del sistema	http://[IP-servidor-staging]:8080
Autenticación en ZAP	Configurar contexto de autenticación con token JWT válido de rol Enfermería y Administrador
Referencia OWASP	OWASP Top 10 – 2021 (A01 a A10)
Momento de ejecución	Al finalizar cada sprint, antes de la Sprint Review

6.2 Casos de prueba DAST

ID	Prueba	Endpoint(s)	Procedimiento	OWASP / Req.	Estado
DAST-01	Autenticación y JWT	POST /api/auth/login	Probar fuerza bruta (100 peticiones/min). Verificar bloqueo de cuenta. Intentar JWT manipulado (payload alterado). Verificar rechazo con HTTP 401.	OWASP A07:2021 – Fallos de identificación	⚠ Pendiente ejecución
DAST-02	Control de acceso por rol	GET /api/medico/cola GET /api/admin/reportes	Intentar acceso a endpoints de Médico con token de rol Admisión. Intentar acceso sin token. Verificar HTTP 403 en todos los casos no autorizados.	OWASP A01:2021 – Control de acceso roto	⚠ Pendiente ejecución

DAST-03	Inyección en API REST	POST /api/pacientes · POST /api/signos-vitales	Enviar payloads de SQL Injection (' OR '1'='1) y XSS (<script>alert(1)</script>) en campos de texto. Verificar que el backend retorna error 400 sin ejecutar el payload.	OWASP A03:2021 – Inyección	⚠ Pendiente ejecución
DAST-04	Manipulación del triaje	POST /api/triaje	Enviar nivel de triaje alterado manualmente (nivel: 1 cuando el algoritmo calculó 4). Verificar que el sistema valida la consistencia del nivel con los parámetros clínicos.	OWASP A04:2021 – Diseño inseguro	⚠ Pendiente ejecución
DAST-05	Privacidad pantalla pública	GET /api/turnos/pantalla-publica	Verificar que la respuesta JSON no incluye nombre completo, documento, EPS ni datos clínicos. Solo iniciales o código de turno (RF-008, Ley 1581/2012).	OWASP A02:2021 – Fallos criptográficos	⚠ Pendiente ejecución
DAST-06	HTTPS obligatorio	Todas las URLs del sistema	Verificar que peticiones HTTP son redirigidas a HTTPS (301). Verificar certificado SSL/TLS válido. Verificar header Strict-Transport-Security presente.	OWASP A02:2021 – Fallos criptográficos	⚠ Pendiente ejecución
DAST-07	Headers de seguridad HTTP	Todas las respuestas	Verificar presencia de: Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy. Ausencia constituye vulnerabilidad media.	OWASP A05:2021 – Configuración incorrecta	⚠ Pendiente ejecución
DAST-08	Rate limiting y DoS	POST /api/auth/login · GET /api/pacientes	Simular 200 peticiones/minuto al login y al listado de pacientes. Verificar que el sistema retorna HTTP 429 (Too Many Requests) antes de degradar el rendimiento.	OWASP A04:2021 / RNF-005	⚠ Pendiente ejecución

6.3 Criterio de aceptación DAST

El sistema MedTriage supera el análisis DAST únicamente si:

- No se detectan vulnerabilidades de severidad CRÍTICA o ALTA en ninguno de los 8 casos de prueba DAST.
- Las vulnerabilidades de severidad MEDIA son documentadas en el Registro de Deuda Técnica del Manual de Mantenimiento con plan de corrección.
- El endpoint de pantalla pública no expone información personal identificable (PII) del paciente.
- El 100% de las peticiones al sistema pasan exclusivamente por HTTPS (verificado con el proxy ZAP).

7. Mapa OWASP Top 10 – 2021

La siguiente tabla mapea cada categoría del OWASP Top 10 (2021) con el estado de cobertura en MedTriage:

Categoría OWASP	Nombre	Mitigación en MedTriage	Amenaza STRIDE	Estado
A01:2021	Control de acceso roto	@PreAuthorize en endpoints + middleware Spring Security + JWT con roles.	AM-E01	Mitigado
A02:2021	Fallos criptográficos	AES-256 en reposo (RNF-001) + HTTPS/TLS en tránsito (RNF-002).	AM-I01, AM-I02	Mitigado
A03:2021	Inyección	JPA parametrizada. Bean Validation en todos los inputs. SpotBugs en SAST.	AM-T02	Mitigado
A04:2021	Diseño inseguro	Validación de consistencia del nivel de triaje con parámetros clínicos en backend.	AM-T01	Mitigado
A05:2021	Configuración de seguridad incorrecta	GlobalExceptionHandler. Headers de seguridad HTTP. Revisión CSRF.	AM-I03, DAST-07	Pendiente Sprint 2
A06:2021	Componentes vulnerables y obsoletos	OWASP Dependency-Check en CI. Plan de parches semestral (Manual Mant.).	SAST-06	Parcial
A07:2021	Fallos de identificación y autenticación	Bloqueo tras 5 intentos. JWT con expiración $\leq 8h$. Verificación presencial reactivación.	AM-S01, AM-S02	Mitigado
A08:2021	Fallos de integridad de software y datos	Firma de código. Verificación de integridad de dependencias en Maven.	—	Pendiente Sprint 4
A09:2021	Fallos en registro y monitoreo	Log de auditoría para triaje modificado, reactivación de usuarios y accesos fallidos.	AM-R01, AM-R02	Pendiente Sprint 2
A10:2021	Falsificación de solicitudes del lado del servidor (SSRF)	No aplicable directamente: MedTriage no realiza peticiones salientes a sistemas externos.	—	No aplicable